Disconnected Perfect Matching on Bipartite Graphs of Radius Three

Research draft · awaiting human expert review
15 September 2026

Abstract
Disconnected Perfect Matching asks whether a graph has a perfect matching whose deletion disconnects it. On bipartite graphs, polynomial-time solvability at radius two and NP-completeness at radius four leave a published open case at radius three. We construct a deterministic Karp reduction from the radius-four case to the radius-three case, with polynomial-time recovery from every target perfect matching that disconnects the output. The construction augments two reference bicliques so that every encoded matching cut extends to a perfect matching. A Boolean encoding preserves the source matching choices, which are recovered from a component of the target after deleting its supplied matching. Outputs have $O(n^3)$ vertices and edges and $O(n^3 \log(n+1))$ bits. The result extends a related Matching Cut construction and establishes the corresponding radius-three hardness boundary, subject to expert validation.

1. Introduction
A matching cut partitions a graph into two nonempty parts while giving each vertex at most one neighbor across the partition. Disconnected Perfect Matching adds an extension requirement: the crossing edges must belong to a perfect matching. This requirement distinguishes the decision problem from both Matching Cut and Perfect Matching Cut, in which the crossing edges themselves cover every vertex.

Lucke [1, Corollary 17] gives a polynomial-time algorithm for Disconnected Perfect Matching on bipartite graphs of radius at most two. The radius-four case is NP-complete [1, Table 1; 3, Theorem 3.3], whereas Section 5, Open Problem 1 asks for the radius-three case. We address this remaining boundary.

Theorem 1. There is a deterministic polynomial-time many-one reduction from Disconnected Perfect Matching on connected simple bipartite graphs of radius at most four to the same problem at radius at most three. Its output has $O(n^3)$ vertices and edges for a source of order n. From every valid target matching, a deterministic polynomial-time algorithm recovers a valid source matching. Consequently, Disconnected Perfect Matching is NP-complete on connected bipartite graphs of radius at most three.

The local binary and selector interfaces come from the related Matching Cut construction [2]. Two enlarged reference bicliques supply the new extension step. Every encoded cut covers all private vertices, while six internal reservoir edges cover the remaining vertices. The source encoding includes actual matching choices so that recovery returns a matching rather than only a vertex coloring. This is a related classification extension, not an independent claim of novelty for the shared interfaces.

2. Witnesses and the Boolean formula
For a connected graph $G = (V, E)$, a witness is a perfect matching $M \subseteq E$ such that $G - M$ is disconnected. Here only edges are deleted. Equivalently, there is a nonconstant coloring $x : V \to \{0, 1\}$ for which every crossing edge belongs to M. In one direction, no edge remaining in $G - M$ crosses the coloring. In the other, color one component of $G - M$ zero and all other components one. This argument also covers deletion into more than two components.

Let the source and target languages be L_S and L_T, with radius bounds four and three respectively. Both use connected simple bipartite graphs. Let $R_S(G, M)$ and $R_T(H, M)$ denote their graph-domain and matching-witness predicates. Graphs are encoded by their integer order and distinct endpoint pairs on labels starting at zero. Malformed encodings are outside the respective language.

For a legal source with $n \geq 2$ vertices and m edges, order the edges lexicographically. Introduce vertex-color variables x_v and edge-selection variables h_e. For each vertex v, write $I(v)$ for its incident edges and impose

$$\bigvee_{e \in I(v)} h_e, \quad \neg h_e \vee \neg h_f \quad (e, f \in I(v), e \neq f). \tag{1}$$

Each unordered pair is used once. For every edge $e = uv$, add

$$(h_e \vee \neg x_u \vee x_v) \wedge (h_e \vee x_u \vee \neg x_v). \tag{2}$$

Finally require both colors by the disjunction of all x_v and that of all $\neg x_v$. Connectedness and $n \geq 2$ ensure that no incident-edge clause is empty.

Lemma 2. The source formula is satisfiable exactly when the source has a disconnected perfect matching. Every satisfying assignment yields such a witness by selecting the edges with $h_e = 1$.

Proof. The constraints in Equation 1 select exactly one edge at each vertex. The constraints in Equation 2 require every crossing edge to be selected. The nontriviality clauses therefore give a perfect matching containing a nontrivial cut. Conversely, a valid matching and the component coloring defined above satisfy all clauses. □

Convert each clause to three-literal disjunctions using fresh chain variables: emit $(\ell_1 \vee \ell_2 \vee z)$ and continue on $(\neg z, \ell_3, \ldots, \ell_k)$ until at most three literals remain. Short clauses repeat their last literal. A true clause extends by letting the chain bit indicate that the processed prefix is false. Eliminating the fresh bits by resolution proves the converse.

Let $E(a, b, c)$ mean exactly one of its three signed literals is true. Replace each three-literal disjunction by fresh variables s_i, t_i and constraints

$$E(s_1, s_2, s_3), \quad E(s_i, \neg \ell_i, t_i) \quad (i = 1, 2, 3). \tag{3}$$

Their equations imply $t_i = \ell_i - s_i$ and $s_i \leq \ell_i$. Selecting the first true literal and setting the slack bits by this equation gives a satisfying extension. Conversely, the selected literal must be true. These are standard Boolean transformations, with independent auxiliary variables for different clauses.

3. Reservoirs and Boolean interfaces

The graph construction uses the at-most-one crossing-neighbor property of a matching cut. A vertex with one opposite neighbor already present is *saturated*: any further incident edge in a valid coloring must be monochromatic.

3.1. Enlarged reservoirs

Create two disjoint complete bipartite graphs with sides

$$U : \{0, 1, 2, u^*\}/\{3, 4, 5\}, \quad W : \{6, 7, 8, w^*\}/\{9, 10, 11\}, \tag{4}$$

and add edge 06. A slash separates biclique sides, not colors. The additional vertices u^*, w^* have only the displayed reservoir neighbors. Figure 1 shows both full reservoirs and the internal matching used later.

Lemma 3 (forcing). Each reservoir is monochromatic in every matching-cut coloring of any graph containing the construction. Any vertex with two distinct neighbors in a reservoir has that reservoir's color.

Proof. An embedded $K_{3,3}$ must be monochromatic. If one side is mixed, its majority of at least two forces the other side to that color. The minority vertex then has three opposite neighbors. Both uniform sides must agree as well. Restricting a coloring to the biclique preserves the upper bound on crossing degree. Each added vertex has three neighbors in that monochromatic biclique, so it agrees. The final claim follows from the two specified neighbors. □

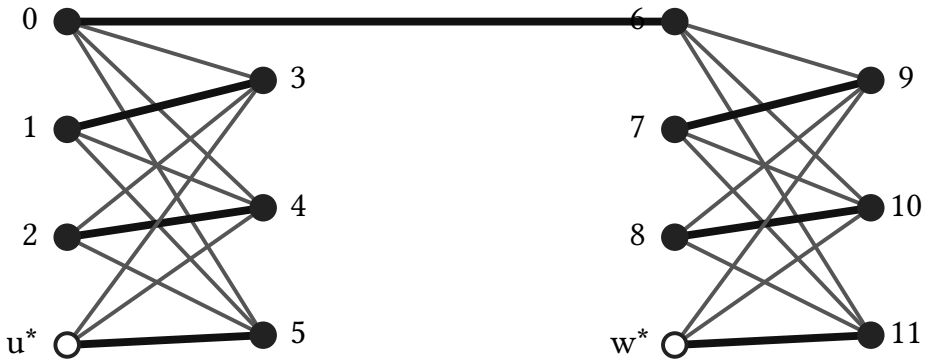


Figure 1: The two $K_{4,3}$ reservoirs, with all fourteen vertices and twenty-five edges shown. Open circles mark the added vertices u^*, w^*. Bold edges are 06 and the six internal matching edges. When the reservoirs have opposite colors, only 06 crosses their colors; the other bold edges cover every reservoir vertex left after its endpoints are removed. Vertex fill indicates roles, not cut colors.

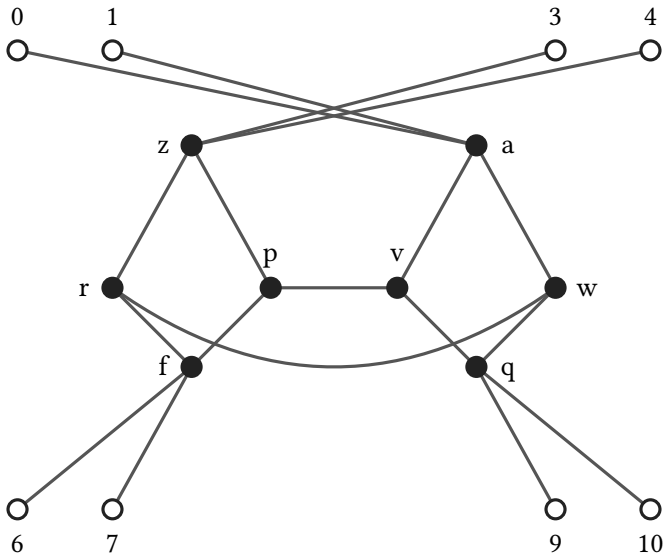


Figure 2: Binary block with eight private vertices and eight displayed reservoir anchors. All eighteen block edges are shown; reservoir-internal edges are omitted and specified in Figure 1. Open circles are anchors. The occurrence ports v, w have complementary colors when the reservoirs differ. The curved rw edge has no intermediate vertex.

3.2. Binary blocks

For each encoded variable, create private vertices f, p, r, z, a, v, w, q and edges

$$\{f6, f7, z3, z4, a0, a1, q9, q10\}, \tag{5}$$

$$\{fp, fr, zp, zr, av, aw, qv, qw, pv, rw\}. \tag{6}$$

Juxtaposition denotes an unordered edge. Figure 2 gives the complete private block and its reservoir attachments.

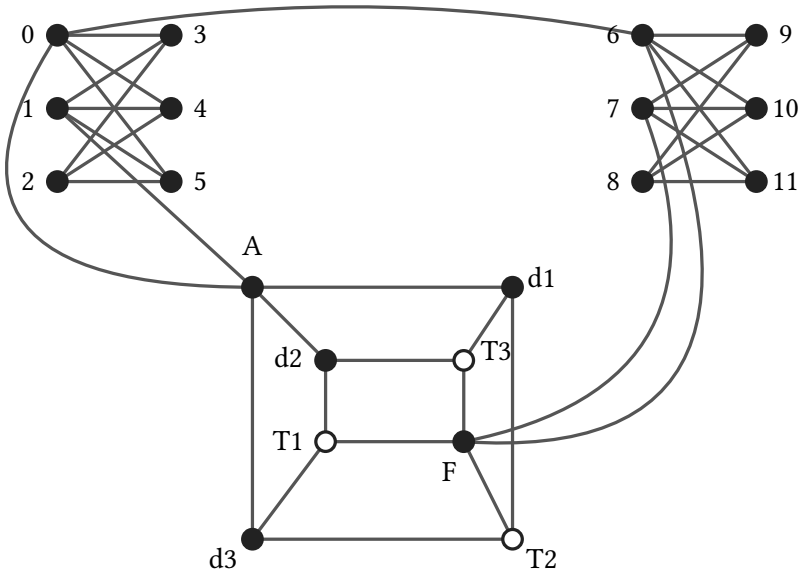


Figure 3: Selector and the twelve original reference vertices: twenty vertices and thirty-five edges shown. The added vertices u^*, w^* and their six edges are omitted here; their full adjacency is in Figure 1. Open circles are high ports T_i. With opposite reference colors, the low bits d_i are one-hot and $T_i = 1 - d_i$. A literal attaches its high port to w for a positive occurrence and v for a negative occurrence in Figure 2; these attachment edges are not shown.

Lemma 4 (binary relation). Normalize the reservoir colors to $U = 0, W = 1$. The binary block has exactly two states,

$$f = q = 1, \quad z = a = 0, \quad p = v = b, \quad r = w = 1 - b, \quad b \in \{0, 1\}. \tag{7}$$

Every private vertex has exactly one opposite private neighbor. If the reservoir colors agree, every private vertex has that common color.

Proof. Lemma 3 fixes the anchor colors. Each of p, r sees anchors of both colors, as does each of v, w, so all four are saturated. The edges pv, rw imply equality. The bounds at a and q force v, w to differ. Each fixed anchor then sees precisely one opposite among its two private neighbors. Either value of b satisfies all bounds. When the reservoirs agree, every free vertex has two anchor neighbors of that color, which forces the same color. □

3.3. Selector blocks

For each signed exact-one triple introduce private vertices $F, A, d_1, d_2, d_3, T_1, T_2, T_3$ and edges

$$\{F6, F7, A0, A1\} \cup \{Ad_i, FT_i : i = 1, 2, 3\} \tag{8}$$

$$\cup \left\{d_i T_j : i, j \in \{1, 2, 3\}, i \neq j\right\}. \tag{9}$$

The high vertices T_i are attachment ports. Figure 3 displays this sixteen-edge block and its reference subgraph.

Lemma 5 (selector relation). If $U = 0, W = 1$, exactly one low bit d_i is one and each $T_i = 1 - d_i$. Every private vertex has exactly one opposite private neighbor. Equal reservoir colors force the whole selector to that common color.

Proof. Lemma 3 gives $A = 0, F = 1$. At most one low bit is one by the bound at A. If none is one, each high bit must be zero because it sees two zero low bits; this gives F three opposite neighbors. Let d_k be the unique one. Its edge to A forces $T_j = 1$ for $j \neq k$. Each other low vertex then sees one of these high bits and forces $T_k = 0$.

The unique internal opposite of T_k is F; the opposite of every other high vertex is the remaining zero low vertex. The other pairs are witnessed by Ad_k, FT_k and the corresponding low-to-high edges. Every one-hot assignment satisfies these bounds. If both reservoirs are zero, any $d_i = 1$ forces two high bits to one, violating F's bound. All low bits, then all high bits, must be zero. Complementation covers common color one. □

4. The complete reduction and recovery

Use one binary block for every Boolean variable and one selector for each exact-one clause. All private labels are distinct between blocks. Attach each high port T_i to the corresponding binary port w for a positive literal and v for a negative literal. The chosen port has the complement of the signed literal's value. Distinct high ports prevent repeated edges even for repeated literals.

Every nonconstant matching cut of this graph has opposite reservoir colors: otherwise Lemmas 4 and 5 force all private vertices to agree with the reservoirs. After normalizing $U = 0, W = 1$, both endpoints of every literal attachment are internally saturated. The edge must be monochromatic, so d_i equals the signed literal value. Thus the cut gives a satisfying exact-one assignment. Conversely, such an assignment extends through both block types with all literal attachments monochromatic. Arbitrarily many occurrences of a variable can therefore share its ports.

Lemma 6 (extension). Every coloring constructed from a satisfying exact-one assignment extends its crossing edges to a perfect matching.

Proof. Lemmas 4 and 5 show that crossing edges cover all private vertices exactly once. Reservoir attachments and literal attachments are monochromatic. The only reservoir crossing edge is 06. The unmatched vertices left in U and W induce two copies of $K_{3,3}$. Add

$$\{(1,3),(2,4),(u^*,5),(7,9),(8,10),(w^*,11)\} \tag{10}$$

to the crossing edges, as shown in Figure 1. These six monochromatic edges cover precisely the remaining vertices. The resulting perfect matching contains the nontrivial cut, so its deletion disconnects the graph. □

The forward map builds the source formula, converts it to signed exact-one constraints and constructs this graph. For labeling, list the original color variables first, then source-edge variables in lexicographic order, then auxiliaries. Binary block i occupies labels $12 + 8i$ through $19 + 8i$ in the order of Lemma 4. Label selectors next and u^*, w^* last. This fixes the source-edge port positions independently of solver choices.

Given any valid target matching M, recompute the output H from the source and find the component S of vertex zero in $H - M$. Return the following source matching, where e_j is the source edge with zero-based index j:

$$g(G, M) = \big\{e_j : 13 + 8(n + j) \notin S\big\}. \tag{11}$$

Only the source and supplied matching are needed. The singleton source is a NO instance and maps to $K_{3,3}$, a legal NO target by Lemma 3. Invalid source strings map to the invalid target encoding `{}`.

Lemma 7 (Karp equivalence and universal recovery). The two maps satisfy

$$G \in L_S \leftrightarrow f(G) \in L_T, \quad R_T(f(G), M) \Longrightarrow R_S(G, g(G, M)). \tag{12}$$

Proof. A source witness gives a satisfying formula by Lemma 2 and extends through the Boolean transformations. Its graph coloring yields a target witness by Lemma 6.

Conversely, let M be any target witness. Its deletion has at least two components. Color S zero and its complement one. Every crossing edge belongs to M, so the crossing degree is at most one. The necessary interface relations therefore give a satisfying exact-one assignment and, by the Boolean transformations, a satisfying source formula. The port rule in Equation 11 selects exactly its source-edge bits, so Lemma 2 proves that the recovered edges form a disconnected perfect matching. This argument imposes no

restriction on the number of deletion components or on the monochromatic matching edges supplied in M. It proves the NO direction as well. The special singleton and malformed cases satisfy equivalence by construction. □

5. Target structure and polynomial bounds

The first side of a bipartition contains $0, 1, 2, u^*, 9, 10, 11$, binary vertices f, z, v, w, and selector vertices F, d_1, d_2, d_3. All remaining vertices lie on the other side. Every listed edge respects these sides, including the literal attachments. Fresh labels ensure simplicity, and every block connects to the joined reservoirs.

Vertex zero reaches vertices $3, 4, 5, 6$ in one step, vertices $1, 2, u^*, 9, 10, 11$ in two steps, and $7, 8, w^*$ in three. Private-vertex distance bounds are:

Block	Distance 1	Distance 2	Distance 3
Binary	a	f, z, v, w	p, r, q
Selector	A	F, d_1, d_2, d_3	T_1, T_2, T_3

These construction paths remain on NO outputs, and literal edges cannot lengthen them. Thus every legal source maps to a connected bipartite target of radius at most three.

For $n \geq 2$, define the neighbor-pair count B, chain-variable count Z and three-literal clause count C by

$$B = \sum_{v \in V(G)} \binom{\deg(v)}{2}, \tag{13}$$

$$Z = \sum_{v \in V(G)} \max(\deg(v) - 3, 0) + 2\max(n - 3, 0), \tag{14}$$

$$C = \sum_{v \in V(G)} \max(\deg(v) - 2, 1) + B + 2m + 2\max(n - 2, 1). \tag{15}$$

The exact-one formula has $N = n + m + Z + 6C$ variables and $K = 4C$ clauses. Direct counting gives

$$|V(f(G))| = 14 + 8N + 8K, \quad |E(f(G))| = 25 + 18N + 19K. \tag{16}$$

Since $m = O(n^2)$ and $B = O(n^3)$, these quantities are $O(n^3)$, and the encoded output uses $O(n^3 \log(n + 1))$ bits. A recovered matching has $\frac{n}{2}$ edges and $O(n \log(n + 1))$ bits.

Both maps use deterministic ordering and the standard library. Source validation checks exact integer types, ordered unique edges, connectivity, bipartiteness and the radius bound. It rejects $n > 2m + 1$ before allocating vertices; connected inputs satisfy this necessary condition. Hence allocated order is bounded by input length even on invalid encodings. Invalid JSON or excessive nesting maps to `{}`; decimal integer parsing has no digit cap. Decimal and binary integer encodings are polynomially equivalent here.

For source encoding length L, the conservative worst-case bit-time bound $O((L + 1)^5)$ covers parsing, validation, clause generation, graph construction and serialization, charging set comparisons in the worst case. Recovery reconstructs the graph, deletes the supplied matching and runs breadth-first search; it uses $O((L + Y + 1)^5)$ bit operations for witness encoding length Y. No runtime solver, randomness or hidden process state is used. Checking a target matching and its deletion connectivity is polynomial, as is checking the target domain. Together with the known source hardness, this proves Theorem 1.

6. Contribution and limitations

The extension settles the radius-three case identified in [1], subject to validation of the draft. Its relation to [2] is explicit: the logical interfaces are reused, while enlarged reservoirs provide a perfect matching

on the residual vertices. Source-edge variables and component-based decoding meet the stronger witness contract.

The literature search of 15 September 2026 found no later exact resolution within its recorded primary-source coverage. Unpublished, unindexed, inaccessible and overlooked work remain possible. This is a related extension of the Matching Cut result; the two constructions should be assessed together when judging publication significance. Agent review and finite tests do not replace human expert assessment.

References

[1] Felicia Lucke. *Matching cut and variants on bipartite graphs of bounded radius and diameter.* Theoretical Computer Science 1052 (2025), 115429. arXiv:2501.08735, Table 1, Corollary 17 and Section 5, Open Problem 1.

[2] *Matching Cut on Bipartite Graphs of Radius Three.* Companion research draft, 15 September 2026. Local manuscript in the preceding Matching Cut campaign; unpublished and awaiting human expert review. Its interface proofs are restated here, so the present argument does not assume the correctness of that draft.

[3] Valentin Bouquet and Christophe Picouleau. *The complexity of the Perfect Matching-Cut problem.* arXiv:2011.03318v2 published paper. Section 2, PDF page 3 defines the historical hyphenated name as the present disconnected-perfect-matching predicate; Theorem 3.3, PDF page 6 gives bipartite diameter-four hardness.

A. Verification and reproduction

Before constructing a candidate, the prepared oracle was compared with exhaustive edge-subset enumeration on all twenty-eight connected bipartite atlas graphs through six vertices, for both source and target. All sixteen valid matching witnesses agreed. Nine stored answers and three deliberately incorrect programs were checked. The rejected programs returned an invalid graph, a wrong decision and an invalid recovered matching.

The unchanged candidate passed nine prepared instances with sixteen recovery calls; its largest prepared output had 4,182 vertices. Four instances were YES and five were NO. A separate author-side implementation checked fifteen instances, seven YES and eight NO, with fourteen recovery calls and maximum output order 3,742. These comprise the connected bipartite atlas graphs through five vertices, a six-cycle, the six-vertex ladder, an eight-cycle and an eight-vertex path. Families overlap, so the suite counts are executions rather than distinct graph totals.

The prepared oracle obtains up to two distinct matchings per YES output and also reverses their edge-list order before recovery. Reversal tests serialization invariance, not a distinct mathematical witness. Both programs solve actual forward outputs, validate the target structure and matchings directly, call recovery in fresh processes and validate the returned source matching. Neither imports the candidate. There was no candidate repair, checker repair, unknown solver result or configured timeout.

The oracle uses Z3 5.1.0 with Boolean edge-selection variables, exactly-one incident-edge constraints and nonconstant cut bits requiring every crossing edge selected. Only conclusive UNSAT is interpreted as NO. The separate general proof establishes arbitrary-size correctness; finite checks do not.

The native independent reviewer advanced without requiring a repair. Its separately authored checker executed the full candidate on $K_{2,4}$ (NO), the cube (YES) and a nine-vertex path (NO). Target orders were 4,030, 5,694 and 3,926. Three distinct solver-found cube-target matchings yielded valid source witnesses. Local exhaustive checks considered all 256 colorings of each eight-vertex block under equal and opposite reference colors, including private-vertex saturation. Four invalid-domain inputs were rejected and the target size formulas were checked. All three recovered target witnesses had two deletion components; unrestricted component counts are covered by the general proof, not by that finite sample. Evidence is in `reviews/complete-rule/`.

From the campaign directory, installed Python 3, `networkx` and `z3-solver` reproduce the checks. The candidate itself needs only the standard library.

```
python3 work/check.py --self-test
python3 work/check.py --candidate work/algorithm.py
python3 work/verify.py --candidate work/algorithm.py
python3 reviews/complete-rule/check.py
```

The two command modes are:

```
python3 work/algorithm.py < source.json > target.json
python3 work/algorithm.py --extract < recovery-input.json
```

A source graph has exactly fields `n` and `edges`, with ordered, distinct integer endpoint pairs. Recovery input has fields `source` and `target_solution`; the latter is the edge list of a valid target perfect matching whose deletion disconnects the target. No target coloring is supplied.

The algorithm and proof are `work/algorithm.py` and `work/proof.md`. Raw prepared and additional results are `rounds/001/prepared.json` and `rounds/001/verification.json`; preparation self-tests are under `work/evidence/preparation/`. Figure generation and exact adjacency checks are in `rounds/001/figures.py`. From `work/`, compile with `typst compile manuscript.typ manuscript.pdf`. The compiled pages were inspected for notation, figure labels, clipping and layout.

Novelty searches combined “Disconnected Perfect Matching” with “radius three”, “radius 3”, “bipartite” and “2026”, followed by the primary locations in [1]. Maximum-size optimization results were distinguished from this decision problem. The review also searched the historical “perfect matching-cut” terminology and checked the predicate and source-hardness theorem directly in [3]. A publisher request for a 2026 induced-path paper failed, so its full argument remains unaudited. Exact queries and coverage limits are retained in the independent review.